

$$\text{GN}(x) = \frac{x - \mu_g}{\sqrt{\sigma_g^2 + \epsilon}} \odot \gamma + \beta$$

Where:

- x : The input feature tensor of shape (N, C, H, W) (Batch, Channels, Height, Width).
- g : The specific group index that a channel belongs to, when channels C are divided into G total groups (so each group contains $\frac{C}{G}$ channels).
- μ_g and σ_g^2 : The mean and variance calculated strictly across all spatial dimensions (H, W) and the subset of channels belonging to group g , computed **independently for each individual sample** in the batch (N) .
- ϵ (epsilon): A small constant added for numerical stability to prevent division by zero.
- γ and β : Learnable scale and shift parameters maintained **per channel** to restore the representation capacity of the network after normalization.

Here is a clear, self-contained Python implementation of **Group Normalization** using PyTorch, matching the mathematical formulation.

```
import torch
import torch.nn as nn

class GroupNorm(nn.Module):
    def __init__(self, num_groups: int, num_channels: int, eps: float = 1e-5):
        super(GroupNorm, self).__init__()
        self.num_groups = num_groups
        self.num_channels = num_channels
        self.eps = eps

        # Learnable scale (gamma) and shift (beta) parameters per channel
        self.gamma = nn.Parameter(torch.ones(1, num_channels, 1, 1))
        self.beta = nn.Parameter(torch.zeros(1, num_channels, 1, 1))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x shape: (N, C, H, W)
        N, C, H, W = x.shape

        # Ensure channels can be evenly split into groups
        assert C % self.num_groups == 0, "Number of channels must be divisible by num_groups"

        # 1. Reshape the tensor to isolate the groups
        # Shape becomes: (N, num_groups, C // num_groups, H, W)
```

```

x_resaped = x.view(N, self.num_groups, C //
self.num_groups, H, W)

# 2. Compute mean and variance across the group channels,
# H, and W dimensions
# We reduce dimensions: (2, 3, 4) which corresponds to
# (C_per_group, H, W)
# Keep dimensions true so broadcasting works cleanly with
# the reshaped tensor
mean = x_resaped.mean(dim=(2, 3, 4), keepdim=True)
var = x_resaped.var(dim=(2, 3, 4), unbiased=False,
keepdim=True)

# 3. Normalize: subtract mean and divide by standard
# deviation + epsilon
x_normalized = (x_resaped - mean) / torch.sqrt(var +
self.eps)

# 4. Reshape back to original dimensions: (N, C, H, W)
x_normalized = x_normalized.view(N, C, H, W)

# 5. Apply learnable affine transformation (gamma and
# beta)
out = x_normalized * self.gamma + self.beta

return out

```

```

# --- Example Usage ---
if __name__ == "__main__":
    # Batch size = 2, Channels = 6, Height = 4, Width = 4
    batch_size, channels, height, width = 2, 6, 4, 4
    num_groups = 3 # Splits 6 channels into 3 groups of 2
    # channels each

    input_tensor = torch.randn(batch_size, channels, height,
width)

    gn_layer = GroupNorm(num_groups=num_groups,
num_channels=channels)
    output_tensor = gn_layer(input_tensor)

    print("Input shape: ", input_tensor.shape)
    print("Output shape:", output_tensor.shape)

```